RPR Automated — Phase 1 Developer Brief

Gaming & Electronics Inventory System — Jectronics LLC

Inventory foundation with FBA location tracking and IronClaw AI agent integration — no external APIs required in Phase 1

v5 — IronClaw AI Agent architecture added. n8n removed. PostgreSQL from day one.

Overview

This brief covers Phase 1 of the RPR Automated multi-phase build. The goal is a working inventory management system built in Python, displayed through a Streamlit dashboard, and stored in a PostgreSQL database. No Amazon or eBay API connections are required in this phase.

The business operates two fulfilment models simultaneously: Amazon FBA (stock shipped to Amazon's warehouse for them to fulfil) and Direct fulfilment (stock held at the business's own warehouse, sold on Amazon or eBay). This distinction is critical throughout the system — it affects inventory tracking, cost of goods, fee structures, and profit reporting. Every part of Phase 1 must be built with this distinction in mind from the start.

> FBA NOTE: FBA stock and Warehouse stock are always your inventory. The difference is physical location. The system must never lose sight of where each unit is at any point in time.

Tech Stack — Phase 1

Tool	Role	Why
Python	Core logic	Connects everything — database, CSV, APIs, Claude
Streamlit	Web app / dashboard	Turns Python into a usable interface — no web dev needed
SQLite	Database	Stores all inventory, shipment, FBA movement and sales data locally
Claude API	AI chat interface	Added in Phase 2 — answers questions over your data
Amazon SP-API	Sales & inventory sync	Added in Phase 3 — automates Amazon data and FBA status updates
eBay Sell API	Sales & inventory sync	Added in Phase 4 — automates eBay data pull
APScheduler	Automation	Runs API syncs on a schedule — hourly or daily
IronClaw	AI Agent layer	Open-source autonomous agent runtime. Runs 24/7 on the cloud server, reads PostgreSQL, reasons over live data using Claude, sends alerts and reports without being asked. Added in Phase 2 after Python foundation is stable.

Inventory Location Model

Every unit of inventory has a location status at all times. Stock always starts as Warehouse when received from a supplier. It moves to FBA In Transit when you ship it to Amazon. It becomes FBA Available when Amazon checks it in. It becomes Sold when a sale depletes it.

Location Status	What It Means	What Triggers It
warehouse	Stock physically held at your location	Default when any supplier shipment CSV is uploaded
fba_in_transit	**Left your warehouse, not yet checked in by Amazon**	**You submit the Log FBA Shipment form**
fba_available	**Checked in at Amazon fulfillment center, available to sell**	**Phase 1: manual confirmation. Phase 3: Amazon SP-API updates this automatically**
sold	Units depleted by a completed sale	Sale recorded — quantity_remaining reduced to zero on that batch

FBA NOTE: In Phase 1, the transition from fba_in_transit to fba_available is triggered manually by the business owner confirming receipt via the dashboard. In Phase 3, the Amazon SP-API will update this automatically.

What to Build — Dashboard Pages

Build these pages in Streamlit, accessible from a sidebar navigation menu. All pages must respect the FBA vs Warehouse distinction.

Page	What It Does	Priority
Upload Shipment	CSV upload — detects new vs existing products, creates FIFO cost layers, sets location to Warehouse	Build first
Current Inventory	Shows all products broken down by location: Warehouse, FBA In Transit, FBA Available. Total across all locations also shown.	Build second
Log FBA Shipment	Record when stock moves from warehouse to Amazon FBA. Moves units from Warehouse to FBA In Transit status.	Build third
Record a Sale	Manual sales entry. Must specify if sale is FBA or Direct. Pulls FIFO cost from the correct location batch.	Build fourth
Shipment History	Log of every inbound shipment from suppliers by date and supplier	Build fifth
FBA Movement Log	Full history of every FBA shipment sent to Amazon, with status and quantities	Build sixth
Sales History	Log of every sale with platform, channel (FBA vs Direct), profit per sale	Build seventh
Alert Settings	Configure alert thresholds — low stock levels, stale inventory days, sales drop percentage. Set recipient email address.	Build eighth

Inbound Shipment CSV Format

The business owner uploads one CSV per supplier delivery. All stock received goes into the Warehouse location by default — no stock ever arrives directly at FBA from a supplier. The system handles three scenarios automatically:

- New product (SKU not seen before) — create product record and warehouse inventory batch
- Existing product, same cost — add quantity to existing warehouse batch
- Existing product, different cost — create new FIFO cost layer in warehouse

Column	Example	Notes
shipment_id	SHIP-2026-001	You assign — one per inbound delivery from supplier
shipment_date	2026-03-08	Date stock was received into your warehouse
sku	RTX4060-EVGA	Your internal code — must be consistent across all records
asin	B08XYZ123	Amazon product ID — used to match FBA stock later
ebay_item_id	123456789012	eBay product ID
product_name	EVGA RTX 4060 8GB	Full product name
category	GPU	GPU, Console, Accessory, etc.
quantity_received	10	Units received — all go to Warehouse location by default
unit_cost	280.00	What you paid per unit — used for FIFO profit calculation
supplier	Ingram Micro	Supplier name
notes	Damaged box on unit 3	Optional — anything worth flagging

On upload, display a confirmation: products created, products updated, new cost layers added. All with location set to warehouse.

Log FBA Shipment Page

This page records when stock leaves your warehouse and is sent to Amazon FBA. It does not record a sale — the stock is still yours, just changing location. This is one of the most important pages in the system.

Field	Example	Notes
Date Shipped to Amazon	2026-03-08	Date you handed stock to carrier
FBA Shipment ID	FBA123456789	Amazon assigns this — find it in Seller Central
SKU	RTX4060-EVGA	Dropdown from products table — only shows SKUs with warehouse stock
Quantity Sent	50	Cannot exceed current warehouse quantity_remaining

Notes	Spring restock	Optional

On submit, the system must:

- Verify the warehouse has sufficient quantity_remaining for the SKU
- Reduce quantity_remaining on the warehouse batch by the quantity sent
- Create a new inventory_batches row with location = fba_in_transit, carrying the same unit_cost from the original warehouse batch (FIFO)
- Create a row in the fba_shipments table for the movement record
- Display confirmation: how many units moved, current warehouse balance remaining

FBA NOTE: The unit cost must travel with the stock. When warehouse batch units move to FBA, the FBA batch inherits the same unit_cost. This ensures profit calculations remain accurate regardless of where the sale originates.

Current Inventory View

The inventory page must show a breakdown by location for every product. Never show only a single total quantity — the business owner needs to see at a glance where every unit is.

Product	Warehouse	FBA Transit	FBA Available	Total	Value
RTX 4060 EVGA 8GB	10	50	0	60	$16,800
PS5 Controller	25	0	100	125	$5,625
Xbox Headset	0	0	30	30	$1,350

The page should also show: total inventory value (sum of quantity_remaining x unit_cost across all batches and locations), a low stock alert for warehouse quantities below a threshold, and an FBA In Transit alert for shipments pending Amazon confirmation for more than 7 days.

FBA NOTE: A product showing 0 Warehouse and 0 FBA Available but 50 FBA In Transit is not available to sell from either channel. The business owner must understand this distinction clearly — the dashboard display must make it unambiguous.

Database Design — PostgreSQL

Use PostgreSQL on the cloud server from day one. IronClaw connects to PostgreSQL natively — SQLite is not compatible with the agent layer. Use SQLAlchemy as the ORM to keep the Python database code clean and portable. Five tables are required:

Table	Key Columns	Purpose
products	sku, product_name, asin, ebay_item_id, category	Master product list — one row per unique product

shipments	shipment_id, shipment_date, supplier, notes	One row per inbound supplier shipment uploaded
inventory_batches	sku, shipment_id, quantity_received, quantity_remaining, unit_cost, location	FIFO cost layers. location column tracks where each batch sits: warehouse, fba_in_transit, fba_available
fba_shipments	fba_shipment_id, date_shipped, sku, quantity_sent, status, notes	Every outbound FBA shipment to Amazon. Links to inventory_batches to track which units moved
sales	sku, platform, channel, sale_price, platform_fee, fba_fee, unit_cost_at_sale, profit	Every sale. channel column distinguishes FBA from Direct. fba_fee column captures Amazon fulfillment fee separately from referral fee

The location Column — Critical

The location column on inventory_batches is the foundation of the entire FBA tracking system. Every query that touches inventory must filter or group by location. Never query raw totals without accounting for location.

FIFO Logic Across Locations

FIFO operates independently within each location. When an FBA sale occurs, the system pulls the unit cost from the oldest fba_available batch for that SKU. When a Direct sale occurs, it pulls from the oldest warehouse batch. The two pools do not mix.

When quantity_remaining hits zero on any batch, that batch is exhausted. The next sale pulls from the next oldest batch of the same location type automatically.

Record a Sale — Manual Entry Form

The Channel field is the most important addition to this form. It determines which inventory pool is depleted and which fee structure applies to the profit calculation.

Field	Example	Notes
Date	2026-03-08	Defaults to today
SKU	RTX4060-EVGA	Dropdown from products table
Platform	Amazon	Dropdown: Amazon / eBay / Other
Channel	**FBA**	**Dropdown: FBA or Direct. Determines which inventory batch to pull from**
Quantity Sold	1	
Sale Price	$349.99	
Amazon Referral Fee	$52.50	Platform commission — applies to both FBA and Direct
FBA Fulfillment Fee	**$5.35**	**Only shown/required when Channel = FBA. Amazon charges per unit fulfilled**
Shipping Cost	$0.00	Only applies when Channel = Direct. FBA shipping is covered by the FBA fee

On submit, the system must:

- Check the Channel field — FBA sales pull from fba_available batches; Direct sales pull from warehouse batches
- Find the oldest batch of the correct location with quantity_remaining greater than zero
- Lock in the unit_cost_at_sale from that batch
- Calculate profit using the correct formula for the channel (see Profit Calculation section)
- Write the sale to the sales table with channel recorded
- Reduce quantity_remaining on the correct batch

FBA NOTE: If the business owner selects Amazon as the platform and FBA as the channel but there are zero units in fba_available, the system must block the sale and display a clear error. It must not silently pull from warehouse stock.

Profit Calculation — By Channel

Profit formulas differ by channel because the fee structures are different. The system must apply the correct formula automatically based on the Channel selected at point of sale.

Sale Type	Formula	Notes
Amazon FBA Sale	**Sale Price - Unit Cost (FIFO) - Referral Fee - FBA Fee**	**Two Amazon fees apply: referral fee (%) + FBA fulfillment fee (flat per unit)**
Amazon Direct Sale	Sale Price - Unit Cost (FIFO) - Referral Fee - Shipping Cost	No FBA fee. Shipping cost is your own fulfillment expense
eBay Sale	Sale Price - Unit Cost (FIFO) - Final Value Fee - Shipping Cost	eBay charges a final value fee instead of a referral fee

```
FBA:    Profit = Sale Price - Unit Cost (FIFO) - Referral Fee - FBA Fulfillment Fee
```

```
Direct: Profit = Sale Price - Unit Cost (FIFO) - Platform Fee - Shipping Cost
```

FBA NOTE: Amazon charges two separate fees on FBA sales: a referral fee (percentage of sale price, typically 8-15%) and an FBA fulfillment fee (flat fee per unit based on size and weight). Both must be stored and displayed separately — never combined — so the business owner can see exactly what Amazon is taking.

Sales Reporting — FBA vs Direct

The Sales History page must always present FBA and Direct sales in separate columns or clearly labelled segments. Never aggregate them into a single figure without also showing the breakdown.

Report View	FBA Column Shows	Direct Column Shows
Revenue	Total FBA sale prices	Total Direct sale prices
Platform Fees	Referral fees + FBA fulfillment fees	Referral/final value fees only

Shipping Costs	$0 — covered by FBA fee	Your fulfillment costs
COGS	Unit cost from FBA batch (FIFO)	Unit cost from warehouse batch (FIFO)
Net Profit	Revenue - COGS - Referral - FBA Fee	Revenue - COGS - Fee - Shipping

Key reports the Sales History page must support in Phase 1:

- Total revenue by channel (FBA vs Direct) for any date range
- Total profit by channel for any date range
- Profit per unit sold, split by channel
- Fee totals by type — referral fees, FBA fulfillment fees, shipping costs — separately itemised
- Best selling products by channel

FBA NOTE: A product might be more profitable sold Direct on eBay than via Amazon FBA once FBA fees are accounted for. The reporting must make this visible. This insight is one of the primary reasons the channel distinction exists in the system.

Alert & Automation System — IronClaw AI Agent

IronClaw is a free, open-source AI agent runtime — the direct successor to OpenClaw, launched February 2026. It runs as a persistent process on the cloud server, connects directly to PostgreSQL, and uses Claude as its reasoning engine. Unlike a fixed workflow tool, IronClaw decides what to investigate and when — making it capable of catching problems nobody explicitly programmed it to look for.

IMPORTANT: IronClaw is introduced in Phase 2, after the Python data foundation is fully built, tested, and validated. The agent is only as reliable as the data it reasons over. Do not attempt to install or configure IronClaw until Phases 1–3 are complete and profit figures have been confirmed accurate by Ryan Gallagher.

Alert Types

Alert Type	Trigger Condition	What the Email Says	Delivery	Severity
Low Stock Warning	Warehouse qty falls below configured threshold (e.g. 10 units)	"RTX 4060 is low — 8 units remaining in warehouse"	Daily check	⚠ Warning
Critical Stockout	Warehouse qty hits zero on any SKU	"Xbox Controller is out of stock in warehouse — 0 units remaining"	Immediate	🔴 Critical
FBA Transit Delay	Units stuck in fba_in_transit for more than 7 days	"FBA shipment FBA123456 has been in transit for 9 days — verify with Seller Central"	Daily check	⚠ Warning
FBA Stock Low	fba_available qty falls below configured threshold	"PS5 Controller has 3 units available at Amazon FBA — consider replenishing"	Daily check	⚠ Warning

Stale Inventory	Item unsold for 30, 60, or 90 days (configurable)	"5 items have not sold in 60+ days — consider repricing or cross-listing"	Weekly report	**i** Info
Sales Anomaly	Revenue drops more than X% vs prior week (configurable)	"Amazon revenue is down 40% vs last week — check listings and inventory"	Daily check	⚠ Warning
Daily Summary	Scheduled — fires every morning regardless of alerts	Revenue today, units sold, top seller, any active alerts listed	Every morning	**i** Info

How the IronClaw Agent Flow Works

```
IronClaw agent process runs continuously on cloud server
  ↓
Agent reads live data from PostgreSQL on a cycle (hourly / daily)
  ↓
Agent reasons over data using Claude API as its intelligence layer
  ↓
Agent decides what conditions warrant action
  ↓
Agent uses tools: email delivery, report generation, database queries
  ↓
Owner receives contextual, reasoned alert or report — not a template
```

Alert Configuration Settings

Alert thresholds are stored in PostgreSQL and read by the agent at runtime. The business owner adjusts them via the RPR Automated dashboard — no agent code changes required.

Setting	Default Value	Configurable By
Low stock threshold — warehouse	10 units	Business owner via Alert Settings page
Low stock threshold — FBA Available	5 units	Business owner via Alert Settings page
FBA transit delay threshold	7 days	Business owner via Alert Settings page
Stale inventory threshold	60 days	Business owner via Alert Settings page
Sales anomaly sensitivity	30% drop vs prior week	Business owner via Alert Settings page
Alert recipient email	Set during setup	Business owner via Alert Settings page
Daily summary send time	7:00 AM	Developer sets in APScheduler config

IronClaw Implementation — What Pak Needs to Build

Component	What It Does	Notes for Developer
IronClaw installation	Installed on cloud server after Phase 3 is complete and validated. Runs as a persistent background process alongside the Python backend.	Install IronClaw only after Python data layer is confirmed accurate. Do not run simultaneously with Phase 1–3 build.
PostgreSQL connection	Agent connects to same PostgreSQL database as the Python backend. Reads inventory, sales, and batch data for its reasoning cycles.	Use read-only database credentials for the agent. Python backend remains the sole writer to the database.
Claude API integration	IronClaw uses Claude as its reasoning engine. Agent sends data context to Claude, Claude reasons and returns decisions, agent acts on them.	Store Claude API key in IronClaw secure config. Monitor token usage — reasoning cycles consume API tokens every cycle.
Tool definitions	Define tools the agent can use: send_email, query_database, generate_report, check_stock_levels. Agent calls these autonomously.	Keep tool definitions narrow and specific. A well-scoped tool is safer and more predictable than a broad one.
Agent goals and memory	Define persistent goals: monitor inventory health, detect sales anomalies, deliver daily summaries, flag margin issues. Memory persists across cycles.	Start with conservative, clearly defined goals. Expand agent scope incrementally after observing behaviour for 2–4 weeks.
Reasoning cycles	Hourly cycle: zero-stock detection only. Daily cycle: full data review, summary generation, threshold checks, report delivery.	Keep hourly cycle lightweight — stockout check only. Daily cycle handles the full reasoning pass with Claude.

All credentials (Claude API key, email auth) are stored in IronClaw's secure configuration — never hardcoded. The agent reads threshold values from the database at runtime so the owner can adjust them from the dashboard without developer involvement.

FBA NOTE: Configure the agent with an hourly reasoning cycle specifically for zero-stock detection on both warehouse and FBA Available. A stockout on a fast-moving item can mean lost sales within hours — the daily cycle is not frequent enough for this condition.

Two-Week Build Plan

Suggested order of work. The FBA location model is integrated from Day 5 so it is never retrofitted.

Days	Task	Outcome

1–2	Install Python, Streamlit, SQLite—get a basic app running in browser	A working Streamlit page loads in the browser

3–4	Build CSV upload page, display uploaded data on screen	Can upload a shipment CSV and see rows displayed
5–6	Connect SQLite, b	Uploaded data persists

	uild_all_database_tables_including_location_column_on_inventory_batch	sists. All tables created with correct schema

7–8	BuildCurrentInventorypagewithWarehouse/FBAInTransit/FBAAvailes	Inventoryviewshowscorrectquantitiesperlocation

	Iablebreakdown	
9–10	BuildLogFBAShipmentpage—movesunitsfromWarehouse	LoggingaFBAshipmentupdatesinventorylocationcor

	toFBAInTransit	rectly
111–112	BuildRecordaSaleformwithFBAvsDirectchannelfield	Salesrecordcorrectly,pullfromrightbatch,calcula

	and separate fee inputs	tep rofit per channel
13–14	BuildSalesHistorypagewithFBAvsDirectbreak	Canseeprofitsplitbychannel.Totalinventory

	down. Add inventory value to totals.	value displayed.
15–16	Build Alert Settings page in dashboard	IronClaw installed and connecting to

	d. Begin IronClaw installation on cloud server. Define initial agent goals	database. Agent goals configured. Basic stock out detection cycle run

	als, tool definitions, and PostgreSQL connection.	ing.
17–18	Connect IronClaw to	Agent reasoning ove

	Claude API. Configure hourly and daily reasoning cycles. Build and test	rlive data. Daily summary email arrives every morning written by Claude

	alalertandreportdeliverybehaviours.End-to-endtesting.	.Allalertsfiringcorrectly.

Installation — Getting Started

What	Command / Action

Install Python	Download from python.org — install version 3.11 or newer
Install Streamlit	pip install streamlit
Install DB + data libraries	pip install sqlalchemy pandas psycopg2-binary
Install IronClaw (Phase 2 only)	Follow IronClaw installation guide at github.com/near-ai/ironclaw — install after Python phase is validated
Test Streamlit works	streamlit hello
Code editor	VS Code — free at code.visualstudio.com. Install Python and Streamlit extensions inside VS Code.

What Comes After Phase 1

- Phase 2 — IronClaw AI Agent: install IronClaw on cloud server after Python data layer is validated. Connect to PostgreSQL and Claude API. Define agent goals, tool definitions, hourly and daily reasoning cycles. Monitor behaviour closely for first 30 days.
- Phase 3 — Claude AI chat interface: wire Claude API into the Streamlit dashboard so the business owner can ask plain-English questions about their own data.
- Phase 4 — Amazon SP-API: automates sales pulls and auto-updates fba_in_transit to fba_available when Amazon confirms receipt. Agent gains richer real-time data to reason over.
- Phase 5 — eBay Sell API: automates eBay Direct sales data. Agent now has full cross-channel visibility for anomaly detection and sourcing insights.

Notes for the Developer

- Build the location column into inventory_batches from Day 1. Do not add it later — it touches every query in the system.
- Every inventory query must filter by location. A raw COUNT of inventory_batches without a location filter is almost always a bug.
- Use PostgreSQL from day one — not SQLite. IronClaw connects to PostgreSQL natively. SQLite is not compatible with the agent layer.
- Use SQLAlchemy as the ORM. This keeps Python database code clean and portable.
- Use pandas for CSV reading — it handles inconsistent formatting gracefully.
- The SKU is the master key. It must be identical across the supplier CSV, the database, and eventually the Amazon and eBay API responses. Agree the SKU format before building anything.
- The business owner has accounting experience and will verify profit numbers. Expect them to catch errors that unit tests will not — build the Sales History page early so they can start checking.
- FBA Fulfillment Fee and Referral Fee must be stored as separate columns in the sales table. Never combine them into a single fee field.
- Do not install IronClaw until Phases 1–3 are complete and profit figures confirmed accurate by Ryan Gallagher. The agent reasons over this data — garbage in, garbage out.
- Give the IronClaw agent read-only database credentials. The Python backend is the sole writer to the database. The agent must never modify data directly.
- Start the agent with narrow, well-defined goals. Expand scope incrementally after observing behaviour for 2–4 weeks. An agent given too broad a remit early is harder to debug.
- Alert threshold values must be stored in the database and read by the agent at runtime — not hardcoded — so the business owner can adjust them from the dashboard without touching the agent.
- All credentials (Claude API key, email auth) go in IronClaw's secure config — never hardcoded in agent definitions or Python files.